

ASSIGNMENT- WEEK7

```
import streamlit as st

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

import plotly.express as px


# Synthetic generation of data examples for training the model

def generate_house_data(n_samples=100):

    np.random.seed(42)

    size = np.random.normal(1500, 500, n_samples)

    price = size * 100 + np.random.normal(0, 10000, n_samples)

    return pd.DataFrame({'size_sqft': size, 'price': price})


# Function for instantiating and training linear regression model

def train_model():

    df = generate_house_data()


    # Train-test data splitting

    X = df[['size_sqft']]

    y = df['price']

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


    # Train the model

    model = LinearRegression()

    model.fit(X_train, y_train)


    return model
```

```
# Streamlit User Interface for Deployed Model
```

```
def main():
```

```
    st.title('🏠 Simple House Pricing Predictor')
```

```
    st.write('Introduce the house size to predict its sale price')
```

```
    # Train model
```

```
    model = train_model()
```

```
    # User input
```

```
    size = st.number_input('House size (square feet)',  
                           min_value=500,  
                           max_value=5000,  
                           value=1500)
```

```
    if st.button('Predict price'):
```

```
        # Perform prediction
```

```
        prediction = model.predict([[size]])
```

```
        # Show result
```

```
        st.success(f'Estimated price: ${prediction[0]:.2f}')
```

```
        # Visualization
```

```
        df = generate_house_data()
```

```
        fig = px.scatter(df, x='size_sqft', y='price',  
                        title='Size vs Price Relationship')
```

```
        fig.add_scatter(x=[size], y=[prediction[0]],  
                        mode='markers',  
                        marker=dict(size=15, color='red'),  
                        name='Prediction')
```

```
        st.plotly_chart(fig)
```

```
if __name__ == '__main__':  
    main()
```